Python: графы

Шпаргалка по структурам данных, стандартной библиотеке, NetworkX и типовым алгоритмам.

Главная идея: для учебных алгоритмических задач чаще всего нужны list/dict/set + deque + heapq. NetworkX удобен для анализа и визуализации, но не всегда доступен в проверяющих системах.

Мини-словарь

Термин	Смысл	Пример
Вершина / node	объект графа	город, пользователь, задача
Ребро / edge	связь между вершинами	дорога, дружба, зависимость
Ориентированный граф	ребро имеет направление u -> v	ссылки, зависимости
Взвешенный граф	у ребра есть стоимость w	время, расстояние, цена
DAG	ориентированный граф без циклов	план задач, prerequisites
Компонента связности	часть графа, где вершины достижимы друг из друга	остров сети

Что запомнить

- V - число вершин, E - число ребер.
- Список смежности обычно лучше матрицы: память O(V + E), а не O(V^2).
- Для невзвешенных кратчайших путей нужен BFS, для весов >= 0 - Dijkstra.

1. Как хранить граф

Список смежности: базовый вариант

```
n, m = map(int, input().split())
g = [[] for _ in range(n)]

for _ in range(m):
    u, v = map(int, input().split())
    u -= 1
    v -= 1
    g[u].append(v)
    g[v].append(u)  # убрать, если граф ориентированный
```

Взвешенный граф

```
g = [[] for _ in range(n)]

for _ in range(m):
    u, v, w = map(int, input().split())
    u -= 1
    v -= 1
    g[u].append((v, w))
    g[v].append((u, w))  # если граф неориентированный
```

Если вершины - строки

```
from collections import defaultdict

g = defaultdict(list)
g["A"].append("B")
g["B"].append("A")
```

Формат	Когда использовать	Память
list[list[int]]	вершины 0..n-1, алгоритмические задачи	O(V + E)
dict/list	строковые ID, произвольные вершины	O(V + E)
матрица смежности	плотный граф или частая проверка u-v	O(V^2)
список ребер	Kruskal, Bellman-Ford, импорт в NetworkX	O(E)

2. Стандартная библиотека

Инструмент	Для чего	Типичные задачи
list / dict / set	базовое хранение графа и visited	почти все задачи
collections.deque	быстрая очередь FIFO	BFS, уровни, путь без весов
collections.defaultdict	автосоздание списков/множеств	граф со строковыми вершинами
heapq	min-heap / очередь с приоритетом	Dijkstra, Prim, A*
graphlib.TopologicalSorter	топологическая сортировка DAG	зависимости задач
itertools	комбинации и пары	перебор возможных ребер

deque: очередь для BFS

```
from collections import deque

q = deque([start])
v = q.popleft()  # достать первый элемент
q.append(to)     # добавить в конец
```

heapq: очередь с приоритетом

```
import heapq

heap = []
heapq.heappush(heap, (distance, vertex))
distance, vertex = heapq.heappop(heap)
```

Не используй list.pop(0) для BFS: список сдвигает элементы. Для очереди почти всегда бери deque.popleft().

3. BFS: обход в ширину

BFS нужен для достижимости, уровней и кратчайшего пути в невзвешенном графе. Сложность: O(V + E).

```
from collections import deque

def bfs_dist(g, start):
    dist = {start: 0}
    q = deque([start])

    while q:
        v = q.popleft()
        for to in g[v]:
            if to not in dist:
                dist[to] = dist[v] + 1
                q.append(to)

    return dist
```

BFS для вершин 0..n-1

```
def bfs_dist_list(g, start):
    dist = [-1] * len(g)
    dist[start] = 0
    q = deque([start])

    while q:
        v = q.popleft()
        for to in g[v]:
            if dist[to] == -1:
                dist[to] = dist[v] + 1
                q.append(to)
    return dist
```

dist[v] == -1 означает, что вершина недостижима. Если dist[target] != -1, путь существует.

4. DFS и компоненты связности

Итеративный DFS

```
def dfs_iterative(g, start):
    visited = set()
    stack = [start]

    while stack:
        v = stack.pop()
        if v in visited:
            continue
        visited.add(v)
        stack.extend(g[v])

    return visited
```

Компоненты связности

```
from collections import deque

def connected_components(g):
    seen = set()
    for start in g:
        if start in seen:
            continue
        comp = []
        seen.add(start)
        q = deque([start])

        while q:
            v = q.popleft()
            comp.append(v)
            for to in g[v]:
                if to not in seen:
                    seen.add(to)
                    q.append(to)
        yield comp
```

Что нужно	Чем делать
все достижимые из start	**DFS или BFS**
количество компонент	**sum(1 for _ in connected_components(g))**
порядок по уровням	**BFS**
глубинный обход дерева	**DFS**

5. Dijkstra: кратчайшие пути с весами

Используй Dijkstra, если все веса ребер неотрицательные: w >= 0. Для отрицательных весов нужен другой алгоритм, например Bellman-Ford.

```
import heapq

def dijkstra(g, start):
    INF = float("inf")
    dist = {start: 0}
    heap = [(0, start)]

    while heap:
        d, v = heapq.heappop(heap)
        if d != dist[v]:
            continue

        for to, w in g.get(v, []):
            nd = d + w
            if nd < dist.get(to, INF):
                dist[to] = nd
                heapq.heappush(heap, (nd, to))

    return dist
```

Шаблон взвешенного списка смежности

```
g[u].append((v, w))
g[v].append((u, w))  # если граф неориентированный
```

Запись if d != dist[v]: continue отбрасывает устаревшие пары из heap. Это обычный прием в Dijkstra на heapq.

6. Topological sort и DSU

graphlib.TopologicalSorter

TopologicalSorter принимает словарь формата node -> predecessors, то есть для каждой вершины указываются те, кто должен идти раньше.

```
from graphlib import TopologicalSorter

deps = {
    "buy": set(),
    "cook": {"buy"},
    "eat": {"cook"},
}

order = list(TopologicalSorter(deps).static_order())
# ['buy', 'cook', 'eat']
```

DSU / Union-Find

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        return True
```

7. Карта выбора алгоритма

Задача	Используй	Сложность
обойти граф	BFS / DFS	O(V + E)
кратчайший путь без весов	BFS	O(V + E)
кратчайший путь, веса >= 0	Dijkstra + heapq	O((V + E) log V)
есть отрицательные веса	Bellman-Ford	O(VE)
все пары кратчайших путей	Floyd-Warshall	O(V^3)
компоненты	BFS / DFS / DSU	O(V + E)
зависимости задач	Topological sort	O(V + E)
минимальное остовное дерево	Kruskal / Prim	зависит от реализации

8. Частые ошибки

- Не добавить обратное ребро в неориентированном графе.
- Использовать list.pop(0) вместо deque.popleft().
- Запустить Dijkstra на отрицательных весах.
- Забыть изолированные вершины.
- Смешать индексацию 1..n и 0..n-1.
- Использовать NetworkX там, где он не установлен или запрещен.

9. NetworkX: быстрый старт

```
pip install networkx matplotlib
```

```
import networkx as nx

G = nx.Graph()      # неориентированный граф
DG = nx.DiGraph()   # ориентированный граф

G.add_node("A")
G.add_edge("A", "B", weight=5)
G.add_edges_from([("B", "C"), ("C", "A")])
```

Тип	Когда использовать
nx.Graph()	**обычный неориентированный граф**
nx.DiGraph()	**ориентированный граф**
nx.MultiGraph()	**несколько ребер между парой вершин**
nx.MultiDiGraph()	**мультиграф + направление**

Просмотр графа

```
list(G.nodes)
list(G.edges)
list(G.edges(data=True))
list(G.neighbors("A"))
G.degree["A"]
G.number_of_nodes()
G.number_of_edges()
G.has_edge("A", "B")
G["A"]["B"]["weight"]
```

10. NetworkX: алгоритмы

Задача	Функции
кратчайшие пути	nx.shortest_path, nx.shortest_path_length, nx.single_source_dijkstra_path_length
обходы	nx.bfs_edges, nx.bfs_tree, nx.dfs_edges, nx.dfs_tree
компоненты	nx.connected_components, nx.weakly_connected_components, nx.strongly_connected_components
DAG	nx.is_directed_acyclic_graph, nx.topological_sort, nx.dag_longest_path
циклы	nx.find_cycle, nx.cycle_basis, nx.simple_cycles
остовное дерево	nx.minimum_spanning_tree, nx.minimum_spanning_edges
центральности	nx.degree_centrality, nx.betweenness_centrality, nx.closeness_centrality, nx.pagerank

Примеры

```
path = nx.shortest_path(G, "A", "C")
length = nx.shortest_path_length(G, "A", "C")

# если есть веса:
path = nx.shortest_path(G, "A", "D", weight="weight")
dist = nx.single_source_dijkstra_path_length(
    G, source="A", weight="weight"
)

# компоненты:
components = list(nx.connected_components(G))

# DAG:
order = list(nx.topological_sort(DG))
```

11. NetworkX: визуализация

```
import matplotlib.pyplot as plt
import networkx as nx

pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos, with_labels=True)

labels = nx.get_edge_attributes(G, "weight")
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)

plt.show()
```

Layout	Когда полезен
nx.spring_layout(G)	**универсальная силовая раскладка**
nx.circular_layout(G)	**вершины по кругу**
nx.shell_layout(G)	**несколько окружностей**
nx.planar_layout(G)	**планарные графы**
nx.spectral_layout(G)	**спектральная раскладка**

12. Что импортировать

```
from collections import defaultdict, deque
import heapq
from graphlib import TopologicalSorter
import itertools

# для анализа и визуализации:
import networkx as nx
import matplotlib.pyplot as plt
```

13. Мини-рецепты

Добавить ребро

```
def add_undirected(g, u, v):
    g[u].append(v)
    g[v].append(u)

def add_weighted(g, u, v, w, directed=False):
    g[u].append((v, w))
    if not directed:
        g[v].append((u, w))
```

Проверить достижимость

```
dist = bfs_dist_list(g, start)
print(dist[target] != -1)
```

Стабильный порядок обхода

```
for neighbors in g:
    neighbors.sort()
```

14. Источники

- **Python docs: graphlib.TopologicalSorter - топологическая сортировка графов.**
- **Python docs: collections.deque - двусторонняя очередь с быстрыми append/popleft.**
- **Python docs: heapq - heap queue / priority queue.**
- **NetworkX docs: tutorial, graph classes, traversal, shortest paths, components,**
- **drawing.**

Полные ссылки использованы при подготовке: docs.python.org/3/library/graphlib.html, docs.python.org/3/library/collections.html, docs.python.org/3/library/heapq.html, networkx.org/documentation/stable/.